Matplotlib & Seaborn MCQ Cheatsheet

1. Matplotlib Fundamentals

1.1 Basic Concepts

What is Matplotlib?

- Low-level Python visualization library
- Provides granular control over plot elements
- MATLAB-inspired syntax
- Works with figures (containers) and axes (plot areas)

Core Objects Hierarchy:

- Figure (top-level container)
    - Axes (individual plot areas)
        - Artists (plot elements: lines, markers, patches)

Two Main APIs:

1. **Object-Oriented (OO) API**: `fig, ax = plt.subplots()` → Direct axis control
2. **Pyplot (Stateful) API**: `plt.plot()` → Implicit current figure/axes

1.2 Installation & Import

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
from matplotlib import cm
```

1.3 Figure & Axes Creation

Single Plot:

```
fig, ax = plt.subplots() # Returns figure and axes objects
fig, ax = plt.subplots(figsize=(10, 6)) # Set size in inches
```

Multiple Subplots:

```
fig, axes = plt.subplots(2, 3) # 2 rows, 3 columns grid
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
ax = axes[0, 0] # Access specific subplot
```

Get Current Axes:

```
ax = plt.gca() # Get current axes
fig = plt.gcf() # Get current figure
```

1.4 Basic Plotting Functions

Line Plot:
ax.plot(x, y) # Default: solid line with markers
ax.plot(x, y, 'r--') # Red dashed line
ax.plot(x, y, linewidth=2, marker='o', color='blue')

Scatter Plot:
ax.scatter(x, y)
ax.scatter(x, y, s=100, alpha=0.5, c='red') # size, transparency, color

Bar Plot:
ax.bar(x, y)
ax.barh(x, y) # Horizontal bar chart
ax.bar(x, y, width=0.5, color=['red', 'blue', 'green'])

Histogram:
ax.hist(data, bins=30)
ax.hist(data, bins=20, edgecolor='black', alpha=0.7)

Box Plot:
ax.boxplot(data)
ax.boxplot([data1, data2, data3], labels=['A', 'B', 'C'])

Pie Chart:
ax.pie(values, labels=labels, autopct='%1.1f%%')

1.5 Styling & Customization

Line Styles & Markers:

- Line styles: `-` solid, `--` dashed, `-.` dash-dot, `:` dotted
- Markers: `o` circle, `s` square, `^` triangle, `*` star, `+` plus, `x` cross
- Format string: 'r--o' = red dashed line with circle markers

Colors:
ax.plot(x, y, color='blue') # Named colors
ax.plot(x, y, color='#FF5733') # Hex colors
ax.plot(x, y, color=(0.1, 0.2, 0.5)) # RGB tuple (0-1 range)

Labels & Title:
ax.set_xlabel('X Axis Label')
ax.set_ylabel('Y Axis Label')
ax.set_title('Plot Title')
ax.set_xlim(0, 100) # Set axis limits
ax.set_ylim(-10, 50)

Legend:
ax.plot(x, y, label='Line 1')
ax.plot(x, y2, label='Line 2')
ax.legend() # Show legend
ax.legend(loc='upper right') # Position: upper right, lower left, etc.
ax.legend(ncol=2) # Number of columns

Grid & Ticks:
ax.grid(True) # Show grid
ax.grid(True, alpha=0.3) # Semi-transparent grid
ax.set_xticks([0, 25, 50, 75, 100])
ax.set_xticklabels(['0%', '25%', '50%', '75%', '100%'])

Figure Layout:
plt.tight_layout() # Auto-adjust spacing
plt.subplots_adjust(left=0.1, right=0.9, top=0.9, bottom=0.1)

1.6 Saving & Displaying

plt.savefig('plot.png', dpi=300, bbox_inches='tight')
plt.show()
plt.close() # Close figure to free memory

---

2. Seaborn Fundamentals

2.1 What is Seaborn?

- High-level visualization library built on Matplotlib
- Statistical data visualization focused
- Better default aesthetics and color palettes
- Works seamlessly with Pandas DataFrames
- Abstracts Matplotlib complexity for common plots

Key Advantage: Less code, more beautiful plots by default

2.2 Installation & Import

import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd

2.3 Setting Style & Context

Styles:
sns.set_theme(style='darkgrid') # darkgrid, whitegrid, dark, white, ticks
sns.set_style('whitegrid')

Palettes (Color Schemes):
sns.set_palette('husl') # husl, Set2, pastel, muted, dark, colorblind
sns.color_palette('coolwarm')

Context (Font Scaling):
sns.set_context('paper') # paper, notebook, talk, poster (increasing size)

Temporary Style Changes:
with sns.axes_style('darkgrid'):
sns.lineplot(data=df, x='x', y='y')

2.4 Axes-Level Functions (Return ax)

Categorical Plots:
sns.scatterplot(data=df, x='col1', y='col2', hue='col3')
sns.lineplot(data=df, x='col1', y='col2', hue='col3')
sns.stripplot(data=df, x='col1', y='col2') # Scatter with jitter
sns.swarmplot(data=df, x='col1', y='col2') # Non-overlapping scatter
sns.barplot(data=df, x='col1', y='col2', hue='col3')
sns.countplot(data=df, x='col1') # Count occurrences
sns.boxplot(data=df, x='col1', y='col2', hue='col3')
sns.violinplot(data=df, x='col1', y='col2') # Distribution shape
sns.pointplot(data=df, x='col1', y='col2') # Line with confidence intervals

Regression & Distribution Plots:
sns.scatterplot(data=df, x='x', y='y')
sns.regplot(data=df, x='x', y='y') # Scatter + regression line
sns.lmplot(data=df, x='x', y='y') # Figure-level (returns figure)
sns.histplot(data=df, x='col', kde=True) # Histogram + KDE
sns.kdeplot(data=df, x='col') # Kernel density estimation
sns.distplot() # DEPRECATED - use histplot or kdeplot

Correlation & Heatmap:
sns.heatmap(data, annot=True, fmt='.2f', cmap='coolwarm')

annot: show values, fmt: number format, cmap: colormap

2.5 Figure-Level Functions (Return figure, more flexible)

sns.relplot(data=df, x='col1', y='col2', hue='col3', col='col4')

Wrapper for scatterplot/lineplot with FacetGrid capability

sns.catplot(data=df, x='col1', y='col2', kind='bar', hue='col3')

kind: bar, box, violin, point, strip, swarm

sns.lmplot(data=df, x='x', y='y', hue='group', col='facet')
sns.displot(data=df, x='col', kde=True, col='group')

2.6 Key Parameters (Across All Plots)

Essential Parameters:

- `data` (DataFrame): Primary input
- `x, y` (str): Column names for axes
- `hue` (str): Color grouping variable (adds legend)

- style (str): Marker style grouping
- size (str): Point size grouping
- col (str): Faceting by columns (separate subplots)
- row (str): Faceting by rows
- palette (str or dict): Color scheme for hue
- ax (matplotlib axis): Target axes for plotting
- height, aspect: Figure-level control (relplot, catplot, etc.)

Statistical Aggregation:
sns.barplot(data=df, x='col1', y='col2', estimator=np.mean) # Default aggregation
sns.barplot(data=df, x='col1', y='col2', ci=95) # 95% confidence interval
sns.barplot(data=df, x='col1', y='col2', errorbar=('sd', 1)) # Custom error bars

2.7 Advanced Seaborn Functions

PairPlot (Scatter Matrix):
sns.pairplot(df) # All numeric columns
sns.pairplot(df, hue='species') # Color by category
sns.pairplot(df, diag_kind='kde') # KDE on diagonal instead of histogram

FacetGrid (Multi-plot Grid):
g = sns.FacetGrid(df, col='col1', row='col2', height=4)
g.map(sns.scatterplot, 'x_col', 'y_col')

JointPlot (Bivariate + Marginal):
sns.jointplot(data=df, x='col1', y='col2', kind='scatter')
sns.jointplot(data=df, x='col1', y='col2', kind='hex') # Hexbin plot
sns.jointplot(data=df, x='col1', y='col2', kind='kde')

ClusterMap (Hierarchical Clustering Heatmap):
sns.clustermap(data, cmap='coolwarm', method='ward')

---

3. Key Differences: Matplotlib vs Seaborn

Aspect	Matplotlib	Seaborn
Level of Abstraction	Low-level, more code required	High-level, less code
Default Aesthetics	Basic/utilitarian	Beautiful, optimized for statistics
Data Input	Arrays, lists, NumPy	Pandas DataFrames preferred
Learning Curve	Steep, detailed control	Gentle, quick results
Plot Types	Wide variety (scatter, bar, line, etc.)	Statistical focus (box, violin, KDE, regression)
Customization	Extensive, granular	Good, but more limited
Stateful API	Yes (plt.plot())	No, requires parameters always
Color Palettes	Limited default palettes	Rich, beautiful default palettes
Integration	Works with any data structure	Seamless Pandas integration
Use Case	General purpose, publication-quality	EDA, statistical visualization

---

4. MCQ Practice Patterns

Pattern 1: Identify Correct Syntax

Q: Which creates a scatter plot with color grouping?

A) `plt.scatter(x, y)` ✓
B) `sns.scatterplot(data=df, x='col1', y='col2', hue='col3')` ✓
C) `ax.scatter(x, y, c=colors)` ✓
D) All of above ✓

Key Insight: Multiple correct answers possible. Know when to use each approach.

Pattern 2: Function Purpose & Output

Q: What does sns.pairplot() return?

A) A Matplotlib axis object ✗
B) A Matplotlib figure object ✓
C) A PairGrid object ✓
D) A list of axes ✗

Key Insight: Figure-level functions return PairPlot/FacetGrid objects, axes-level return axes.

Pattern 3: Parameter Effects

Q: What does the `hue` parameter do in Seaborn?

A) Creates subplots ✗
B) Colors data by a third variable ✓
C) Changes figure size ✗
D) Adds a regression line ✗

Pattern 4: Default Behaviors

Q: What is the default aggregation function in sns.barplot()?

A) Median ✗
B) Sum ✗
C) Mean ✓
D) Count ✗

Key Insight: Seaborn uses statistical defaults (mean with CI).

Pattern 5: Customization Order

Q: Which approach correctly customizes a subplot?

Correct: Use the returned axes object

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title('Title')
ax.set_xlabel('X')
```

Key Insight: OO approach gives explicit control. Stateful API implicit.

---

5. Common Tricky Concepts

5.1 Figure vs Axes

INCORRECT: Confusing figure and axes

```
plt.title('Plot') # Sets on current figure
plt.plot(x, y)
ax = plt.gca()
ax.set_title('Plot') # Sets on current axes (correct for OO)
```

CORRECT: Use OO approach consistently

```
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title('Plot')
```

5.2 Seaborn Hue vs Matplotlib c Parameter

Seaborn: Use hue parameter

```
sns.scatterplot(data=df, x='x', y='y', hue='category')
```

Matplotlib: Use c parameter

```
ax.scatter(x, y, c=colors) # colors must be numeric or RGB
```

5.3 Figure-Level vs Axes-Level

Axes-level: Works with ax parameter

```
ax = plt.gca()
sns.scatterplot(data=df, x='x', y='y', ax=ax)
```

Figure-level: Creates own figure, can't use ax

```
fig = sns.relplot(data=df, x='x', y='y') # Returns figure
```

5.4 Stateful vs Stateless

Matplotlib stateful (implicit)

plt.plot(x, y) # Plots on current axes
plt.title('Title') # Title on current figure

Seaborn always stateless (explicit)

sns.scatterplot(data=df, x='x', y='y') # Needs data and column names

Cannot do: sns.scatterplot(x, y) # Will fail

5.5 Deprecated vs Current

DEPRECATED

sns.distplot(data) # Don't use

CORRECT

sns.histplot(data, kde=True) # Use histplot
sns.kdeplot(data) # Use kdeplot separately

---

6. Quick Reference: Common Plot Types

When to Use:

- **Scatter**: Show relationship between 2+ continuous variables
- **Line**: Show trends over time or continuous variable
- **Bar**: Compare values across categories
- **Histogram**: Show distribution of single continuous variable
- **Box Plot**: Show distribution, quartiles, and outliers
- **Violin Plot**: Show full distribution shape (better than box for multimodal)
- **Heatmap**: Show 2D matrix with color intensity
- **Regression Plot**: Show linear relationship with confidence interval
- **KDE**: Smooth probability density estimation

---

7. Critical MCQ Questions

Q1: What's the output of plt.subplots(2,3)?

- A single figure with 6 subplots arranged 2×3 ✓

Q2: Can Seaborn work without Matplotlib?

- No, Seaborn is built on top of Matplotlib ✓

Q3: Which is stateless (requires column names)?

- Seaborn (sns.plot() requires column names from DataFrame) ✓

Q4: How to add a custom axis label in Seaborn?

- Use Matplotlib: `ax.set_xlabel()` after plotting ✓

Q5: What does plt.tight_layout() do?

- Automatically adjusts subplot spacing to prevent overlaps ✓

Q6: Can you use hue parameter in Matplotlib?

- No, use 'c' parameter instead for color grouping ✓

Q7: Which plot handles categorical × continuous best?

- Violin plot or Box plot for distribution ✓
- Bar plot for aggregated statistics ✓

Q8: Default confidence interval in sns.barplot()?

- 95% CI based on bootstrap ✓

Q9: What does sns.set_palette() do?

- Sets default color palette for all subsequent plots ✓

Q10: Difference between sns.lineplot() and sns.lmplot()?

- lineplot() is axes-level, lmplot() is figure-level ✓

---

8. Code Snippet Flashcards

Create simple scatter plot:

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots()
ax.scatter([1,2,3], [4,5,6])
plt.show()
```

Create scatter with grouping:

```
import seaborn as sns
sns.scatterplot(data=df, x='col1', y='col2', hue='category')
plt.show()
```

Create multi-panel visualization:

```
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
axes[0, 0].plot(x, y)
axes[0, 1].scatter(x, y)
axes[1, 0].bar(x, y)
axes[1, 1].hist(data)
plt.tight_layout()
```

Set global style:

```
sns.set_theme(style='darkgrid', palette='husl')
```

All subsequent plots use this style

---

9. Exam Tips

1. **Know the objects**: Figure (container) → Axes (plot) → Artists (elements)
2. **OO approach is safer**: `fig, ax = plt.subplots()` then `ax.method()`
3. **Seaborn needs DataFrames**: Always pass `data=df` parameter
4. **Axes-level vs Figure-level**: Axes work with `ax=`, figure-level don't
5. **Color parameters differ**: Matplotlib uses `c=`, Seaborn uses `hue=`
6. **Stateful vs Stateless**: Matplotlib has both, Seaborn only stateless
7. **Customization**: Use Matplotlib functions on Seaborn plots for tweaking
8. **Performance**: Both similar speed, Seaborn faster for complex statistical plots
9. **Deprecated**: `sns.distplot()` → use `sns.histplot()` or `sns.kdeplot()`
10. **Read the error messages**: Wrong parameter names are caught immediately